

# **Brieffreund**

„Brieffreund“ ist ein Projekt von Thomas Greshake, welches die Gangfolge von Postzustellern berechnet und verbessern soll. Der Code ist auf [Github](#) zu finden. Das Konsolen-Programm für Windows versucht die folgenden Aspekte zu optimieren:

## **Ablagen**

Ablagen werden von Beginn berücksichtigt und an regelmäßigen Stellen und in der gewünschten Anzahl in die Gangfolge eingebaut. Durch Ablagen entstehende Umwege werden minimiert.

## **Sicherheit**

Das unnötige Überqueren von stark befahrenen Straßen wird minimiert. Allgemein wird das Fahren auf der richtigen Straßenseite bevorzugt. Außerdem wird versucht, Einbahnstraßen, welche auch für Fahrräder gelten, in der richtigen Richtung zu durchfahren.

## **Kürze**

Die Gesamt-Route sollte so kurz wie möglich sein. So wird unnötige Arbeitszeit vermieden und die Abnutzung der Betriebsmittel minimiert.

## **Leserlichkeit**

Die fertige Route sollte vom Briefzusteller nachvollziehbar sein.

## **Modularität**

Die Angabe einer festen Start- und Endadresse der Gangfolge für die Eingliederung in eine Gesamtgangfolge ist möglich, aber nicht zwingend.

Das Programm kommt all diesen Vorgaben (innerhalb der später diskutierten Limitationen) tatsächlich sehr gut nach. Die durchschnittliche berechnete Gangfolge ist fast 1 km kürzer, hat eine bessere Postverteilung über die Ablagen, benutzt öfter die richtige Straßenseite und überquert dabei viel befahrene Straßen seltener. Genaue Werte werden später diskutiert.

## **Schnellstart**

Damit das Programm erfolgreich sein kann, müssen die „Bezirks-Datei“ und die „Osm-Datei“ sich im selben Ordner wie „Brieffreund.exe“ befinden. Eine Auswahl an Bezirks-Dateien und den zugehörigen Osm-Dateien ist bereits zur Verfügung gestellt. Dasselbe gilt für alle mitgelieferten „Dynamic Link Libraries“ (.dll).

### **Bezirksdatei**

Dies ist eine simple Textdatei (.txt) mit der Beschreibung des Bezirks, dessen Gangfolge erstellt werden soll. Zuzustellende Adressen, Ablagen, die zu verwendende Osm-Karte, sowie Start- und Endadresse sind hier aufgelistet.

### **Osm-Datei**

Dies ist eine [Open-Street-Map](#) Karte (.osm). Sie enthält die Informationen über die Lage der örtlichen Straßen und Adressen. Es wird ausschließlich das osm-Format unterstützt!

Um das Programm auszuführen, reicht ein Doppelklick. Die Konsole öffnet sich automatisch. Danach fehlen nur noch zwei Schritte:

#### **1. Angabe der Bezirksdatei**

„Enter“ bestätigt automatisch den Standardnamen („Bezirk.txt“), aber eine beliebige beigefügte oder selbst erstellte Datei ist ebenfalls möglich (Bezirk\_72, Bezirk\_73, Bezirk\_75\_Alt, Bezirk\_77, Bezirk\_78, Bezirk\_79, Bezirk\_80, Bezirk\_80\_Alt, Bezirk\_81, Stresstest\_73\_81, Stresstest\_77\_79).

#### **2. Eingabe der Suchtiefe**

„Enter“ bestätigt automatisch den Standardwert (16). Suchtiefe hat einen abnehmenden positiven Effekt, sodass höhere Werte meist sinnlos sind. Für schlechtere Computer oder aus sonstigen Gründen auftretende Rechenzeit-Probleme reicht „4“ locker aus.

Da die beigefügten Bezirksdateien alle restlichen Informationen liefern, beginnt danach bereits die Berechnung einer Gangfolge. Abschließend wird das Ergebnis mitsamt einer Analyse in der Konsole angezeigt. Soll die erstellte Gangfolge gespeichert werden, muss ein Name für die Speicher-Datei angegeben und mit „Enter“ bestätigt werden. Wird dies nicht getan, verwirft „Enter“ das Ergebnis. Die abgespeicherte Datei ist unter „Dokumente/Brieffreund/Speichername“ zu finden.

## **Analyse der Gangfolge**

Die erstellte Route wird automatisch analysiert. Wenn die Bezirksdatei die zuzustellenden Adressen in Gangfolge angibt, so kann die erstellte Route sogar mit der alten Route verglichen werden. Die Zahlen in der Analyse haben folgende Bedeutung:

### **Straßenlänge**

Die Gesamtlänge der Gangfolge in Metern. Dies beinhaltet auch die Strecken von und zu den Ablagen. Straßenüberquerungen sind nicht in dieser Zahl berücksichtigt.

### **Überquerungsgewicht**

Jede überquerte Straße addiert ein Gewicht zu diesem Wert, welches eine Kombination aus der Breite und Verkehrsstärke der Straße darstellt.

### **Gesamt(länge)**

Die Summe aus Straßenlänge und Überquerungsgewicht. Es ist sozusagen die Gesamtlänge des Bezirks in Metern, inklusive Straßenüberquerungen, wenn schwer befahrene Straßen viel breiter aber dafür nicht befahren wären.

### **Falsche Straßenseite**

Die Länge in Metern, bei der die Zustellung entgegen der Fahrtrichtung erfolgt. Ob dies tatsächlich ein Problem ist, hängt stark von den lokalen Bedingungen ab.

### **Gegen Einbahnstraße**

Die Länge in Metern, bei der die Zustellung entgegen einer Einbahnstraße erfolgt. Ob dies tatsächlich ein Problem ist, hängt stark von den lokalen Bedingungen ab. Da manche Einbahnstraßen in Open-Street-Map nicht korrekt bezeichnet sind (Viele sind in Wahrheit für Fahrräder erlaubt, in der Karte aber als verboten bezeichnet), ist die echte Zahl vermutlich kleiner als die angegebene.

### **Anzahl an Straßenüberquerungen, sortiert nach Verkehrsstärke**

Selbsterklärend. Überquerungsgewicht ist bereits eine gute Zusammenfassung dieser Zahlen.

### **Postverteilung: Standardabweichung**

Mit der angegebenen Zahl an gewünschten Ablagen wird eine gewünschte durchschnittliche Postmenge je Ablage ermittelt. Der angegebene Wert ist die [Standardabweichung](#) der erreichten Verteilung der Postmenge von diesem Durchschnittswert (Vorsicht: NICHT vom Mittelwert der vorliegenden Verteilung, wie dies normalerweise in der Stochastik der Fall ist), dividiert durch denselben. Grob gesagt: Eine Standardabweichung von 30% bedeutet, dass die Postmenge je Ablage durchschnittlich um 30% vom gewünschten Wert abweicht, unabhängig davon, ob dies nach unten oder oben passiert. Kleinere Werte sind besser.

### **Postverteilung: Max. positive Abweichung**

Die maximale Abweichung der Postmenge einer Ablage vom gewünschten Wert nach oben. Je größer dieser Wert, desto voller ist die größte Ablage im schlimmsten Fall. Wann eine Abweichung zu hoch ist, hängt natürlich stark davon ab, wie viele Ablagen im voraus eingeplant werden und welches Betriebsmittel genutzt wird. Das Programm geht intern davon aus, dass im Extremfall eine 100% Abweichung nach oben gerade noch okay ist.

### **Postverteilung: Max. negative Abweichung**

Die maximale Abweichung der Postmenge einer Ablage vom gewünschten Wert nach unten. Je größer dieser Wert, desto leerer (und damit unnötiger) ist die kleinste Ablage.

## Vergleich zweier Gangfolgen

Mit der Analyse lässt sich die erstellte Gangfolge auch mit einer alten Gangfolge vergleichen. Hierzu muss die Bezirks-Datei die alte Gangfolge wiedergeben. Dies funktioniert allgemein sehr gut. Trotzdem ein Wort der Warnung: Das Programm vergleicht die eigene Gangfolge mit der ursprünglichen „im eigenen Kontext“, wodurch ein kleiner unvermeidbarer Bias entsteht, sollte die Gangfolge mit einer externen, nicht vom Programm erstellten, Gangfolge verglichen werden.

Ein Beispiel: Der Eingang eines Hauses liegt nicht dort, wo man ihn vermuten würde. Das Programm glaubt ihn nun an einer falsche Stelle und sortiert den Eingang damit auch an die falsche Stelle in seiner Gangfolge. In der ursprünglichen Gangfolge liegt der Eingang jedoch an der tatsächlich richtigen Stelle, was aus Sicht des Programms aber einen unnötigen Umweg darstellt.

Für jeden angegebenen Wert kann man ein theoretisches Beispiel finden, das den Vergleich leicht verfälscht. Der Fehler ist von seiner Natur aus unvermeidbar. Mit Ausnahme von Extremfällen sollte der Effekt allerdings klein sein.

### **Vergleich der berechneten Gangfolgen mit den alten Gangfolgen 2025/2026**

Die beigegeführten Dateien „Bezirk“, „Bezirk\_72“, „Bezirk\_73“, „Bezirk\_77“, „Bezirk\_78“, „Bezirk\_79“, „Bezirk\_80“ und „Bezirk\_80\_Alt“ sind in der Gangfolge, wie sie von den Zustellern im Jahrgang 2025/2026 tatsächlich gelaufen wurden, und werden automatisch verglichen („Bezirk“ ist Bezirk 75 und Bezirk\_80\_Alt stammt aus 2024/2025).

Mit Ausnahme von „Bezirk\_80\_Alt“, welcher sich zu sehr mit „Bezirk\_80“ überschneidet, werden hier ein paar Daten des Vergleichs zwischen den alten und neuen, vom Programm berechneten, Gangfolgen der restlichen 7 Bezirke aufgelistet.

|                          | Alte Routen | Neue Routen | Differenz | Änderung |
|--------------------------|-------------|-------------|-----------|----------|
| Straßenlänge (m)         | 7472        | 6497        | -975      | -13%     |
| Überquerungsgewicht      | 571         | 565         | -6        | -1%      |
| Gesamtlänge              | 8043        | 7062        | -981      | -12%     |
| Falsche Straßenseite (m) | 1929        | 1011        | -918      | -48%     |
| Gegen Einbahnstr. (m)    | 265         | 178         | -87       | -33%     |
| SL Überquerungen         | 32          | 35,4        | +3,4      | +11%     |
| Überquerungen > SL       | 8,7         | 6,1         | -2,6      | -30%     |
| Postverteilung $\sigma$  | 40%         | 36%         | -4%       | -10%     |

Alle Werte sind die Durchschnittswerte über die 7 betrachteten Bezirke. „SL Überquerungen“ ist die Anzahl an Überquerungen von Straßen mit „sehr leichtem Verkehr“, während „Überquerungen > SL“ die Anzahl von Überquerungen aller anderer Straßen mit schwererem Verkehr ist. „Postverteilung  $\sigma$ “ ist die zuvor beschriebene Standardabweichung der Postverteilung.

Die vom Programm berechneten Gangfolgen sind durchweg kürzer und überqueren viel befahrene Straßen seltener. In manchen Fällen wird die Zustellung auf der falschen

Straßenseite extrem reduziert. Die Ablagen sind für jeden Bezirk regelmäßig einsortiert, ähnlich der alten, von Hand eingetragenen Ablagen, und haben sogar eine bessere Postverteilung mit geringerer Standardabweichung.

## **Erstellung einer eigenen Bezirks-Datei**

Das Programm benutzt momentan ein von mir erstelltes Format für die Textdatei, welche den Bezirk enthält. Dieses Format wird hier erklärt, sodass der Benutzer jeden möglichen Bezirk berechnen lassen kann.

### **Allgemeines**

Die Datei ist eine normale Text-Datei (.txt). Man erhält bereits einen guten Überblick, indem man eine beigefügte Bezirks-Datei einfach öffnet und betrachtet. Kommentare sollten die Zeile mit „#“ beginnen. Es muss darauf geachtet werden, dass alle Straßennamen richtig geschrieben werden. „Karl-Bart-Str.“ (falsch) statt „Karl-Barth-Str.“ (richtig) wird z.B. nicht erkannt. „Karl Barth Straße“ wäre hingegen okay. Im Zweifel sollte der Straßename [geprüft](#) werden.

### **Postleitzahl und Stadt**

Die Postleitzahl und Stadt des Bezirks muss angegeben werden, hierzu muss die Zeile mit „Plz“ bzw. „Stadt“ beginnen. Erlaubt sind mehrere Einträge in einer Zeile, z.B. „Plz 53129, 53113“ oder „Stadt Bonn, Köln, Düsseldorf“. Zuzustellende Adressen müssen in der PLZ und Stadt Kombination eindeutig sein.

Beispiel:

|       |       |
|-------|-------|
| Plz   | 53129 |
| Stadt | Bonn  |

### **Osm-Karte**

Eine Open-Street-Map Karte, in der der Bezirk vollständig enthalten ist, kann unter „Osm“ angegeben werden. Wenn keine Angabe gemacht wird, wird diese Info in der Konsole abgefragt. Neue Karten können für alle beliebigen Orte [hier](#) exportiert werden (Zoom auf den Bezirk, dann „Export“). Die Karte muss im osm-Format sein.

Beispiel:

|     |         |
|-----|---------|
| Osm | map.osm |
|-----|---------|

### **Start- und Endadresse**

Eine Start- und Endadresse kann mit „Start“ bzw. „Ende“ angegeben werden. Diese Info wird nicht exakt beachtet, der tatsächliche Start- und Endpunkt liegt aber sehr nah. Start und Ende müssen Adressen sein, die auch tatsächlich Post erhalten (ansonsten werden sie den Postadressen hinzugefügt). Start und Ende müssen nicht angegeben werden, das Programm wird bei Nicht-Angabe nach geeigneten Adressen suchen.

Beispiel:

|       |                    |
|-------|--------------------|
| Start | Karl-Barth-Str. 34 |
|-------|--------------------|

### Postadressen

Zuzustellende Adressen müssen die Zeile mit „Adr“ beginnen, gefolgt von dem Straßennamen und einer Auflistung der zuzustellenden Hausnummern, die Reihenfolge ist egal und hat keinen Einfluss auf die Funktionsweise des Programms. Soll die berechnete Gangfolge später verglichen werden, so muss die Eingabe in Gangfolge sein. In diesem Fall können auch Ablagen mit „AdrAbl“ in der Gangfolge angegeben werden.

Beispiel:

|        |                 |               |
|--------|-----------------|---------------|
| Adr    | Karl-Barth-Str. | 1, 2, 3, 4, 5 |
| Adr    | Karl-Barth-Str. | 7, 8, 9       |
| AdrAbl | Markusplatz     | 1             |

Vorsicht: „AdrAbl“ dient ausschließlich dem Vergleich zweier Gangfolgen mit Ablagen, die korrekte Angabe von Ablagen folgt.

### Ablagen

Adressen von Ablagen müssen mit „Abl“ beginnen, gefolgt von der maximal (!) verfügbaren Anzahl an Ablageteilen (Teil 1, Teil 2, ..., basierend auf der Größe des Ablagekastens), gefolgt von der Adresse. Es müssen keine Ablagen angegeben werden, und falls keine Ablagen angegeben sind, wird einfach nur die kürzeste Route ohne Ablagen berechnet. Idealerweise sollten die Adressen nicht nur die Position, sondern auch die Straßenseite der Ablage so gut wie möglich widerspiegeln. Achtung: Dies ist bei manchen Ablagen-Adressen im System momentan nicht der Fall!

Beispiel:

|     |    |                    |
|-----|----|--------------------|
| Abl | 3x | Bergstr. 3         |
| Abl | 4  | Burbacher Str. 195 |

Die Ablage an der Bergstr. 3 kann bis zu dreimal benutzt werden, während die Ablage in der Burbacher Str. 195 bis zu viermal benutzt werden kann. Man bemerke, dass das „x“ optional ist.

### Ablagen-Anzahl

Die ideale Anzahl an Ablagen, bei der die Postmenge gut verteilt ist, kann mit „Anz“ angegeben werden. Vorsicht: Das Anfangsstück wird nicht mitgezählt. Die Angabe wird nur benötigt, wenn Ablagen angegeben sind, sonst wird einfach die kürzeste Route berechnet. Genau dasselbe passiert, wenn die Anzahl mit 0 angegeben ist.

Beispiel:

|     |   |
|-----|---|
| Anz | 6 |
|-----|---|

### Analyse

Die resultierende Gangfolge kann mit „Analyse ja“ auf verschiedene Daten analysiert werden. Dies passiert auch automatisch, wenn keine Angabe gemacht wird, „Analyse

nein“ unterdrückt die Funktion. „Analyse Vergleich“ vergleicht die erstellte Gangfolge mit der Gangfolge in der Bezirks-Datei. Erinnerung: Hierzu sollten Ablagen mit „AdrAbl“ in der Gangfolge erwähnt werden.

Beispiel:

Analyse            Vergleich

### **Verkehrsstärke**

Straßen können, aber müssen nicht, mit einer Verkehrsstärke assoziiert werden. Die Zeile muss mit "Verkehr" beginnen, gefolgt vom Straßennamen, gefolgt von der Verkehrsstärke. Für Straßen, die hier nicht angegeben sind, wird das Programm automatisch eine Verkehrsstärke finden. Dies funktioniert allgemein sehr gut, kann hier aber manuell überschrieben werden. Die Verkehrsstärke kann die folgenden Werte haben: 0 - Sehr Leicht, 1 - Leicht, 2 - Mittel, 3 - Schwer, 4 - Sehr schwer. Das Programm wird versuchen, Straßen mit hohem Verkehr seltener zu überqueren, ferner kommt eine Straßen nur für „Links-Rechts-Zustellung“ in Frage, wenn die Verkehrsstärke „Sehr leicht“ ist. Sollte eine Straße für Links-Rechts-Zustellung aus diversen Gründen ungeeignet sein, so kann die Verkehrsstärke ohne Bedenken auf „Leicht“ gesetzt werden, selbst wenn die Verkehrsstärke in Wahrheit „Sehr leicht“ ist.

Beispiel:

Verkehr            Karl-Barth-Str.            3

Der Verkehr auf der Karl-Barth-Str. ist schwer (3).

### **Suchtiefe**

Die Suchtiefe kann mit „Suchtiefe“, gefolgt von der gewünschten Zahl, modifiziert werden. Eine Erhöhung der Suchtiefe führt zu etwas längerer Rechenzeit, aber nicht unbedingt zu besseren Resultaten. Muss nicht angegeben werden.

Beispiel:

Suchtiefe            8

### **Löschen einer Straßen-Verbindung**

Eine Straßen-Verbindung zwischen zwei Adressen kann mit „Rem“ (Remove) gelöscht werden, z.B. wenn eine Gasse zu eng für das Betriebsmittel ist oder der Weg blockiert ist. Diese Implementierung ist natürlich ungenau, falls die Adressen weit entfernt liegen. Aber bis zur Einführung einer grafischen Benutzeroberfläche gibt es keine bessere Lösung (außer die osm-Karte direkt zu modifizieren, z.B. mit [josm](#)). Das Format ist: Rem *erste Adresse* : *zweite Adresse*.

Beispiel:

Rem            Burbacher Str. 143 : Füssenichgasse 1

Die Verbindung zwischen der Burbacher Straße 143 und der Füssenichgasse 1 wird gelöscht.

## Argumente

Das Programm kann auch in der Konsole gestartet werden. Der dem Programmnamen direkt folgende Text muss dann der Name der gewünschten Bezirks-Datei sein. Außerdem gibt es die folgenden Argumente:

- k, --karte: Name der zu nutzenden osm-Datei.
- s, --suchtiefe: Die gewünschte Suchtiefe.
- a, --ablagen: Die Anzahl der gewünschten Ablagen.
- g, --gasse: Gefolgt von „erlaubt“ wird das Durchqueren schmaler Gassen erlaubt, gefolgt von „stopp“ ist dies nicht erlaubt. Standard ist „stopp“. Das Erkennen schmaler Gassen ist sehr ungenau.
- v, --vergleich: Hiermit wird die berechnete Gangfolge mit der aktuellen verglichen.

Gibt es sich überschneidende Angaben zwischen der Bezirks-Datei und den Argumenten, so gewinnen die Argumente immer (Man könnte sagen, sie sind einfach besser).

### **Beispiel**

„./Brieffreund Bezirk\_72 -s 8 -a 4 -v“ in der Konsole (welche im Ordner geöffnet ist) berechnet die Gangfolge des Bezirks 72, mit der Suchtiefe 8, sortiert 4 Ablagen ein und vergleicht das Ergebnis mit der aktuellen Gangfolge in der Bezirks-Datei.

## Funktionsweise

Hier wird die Funktionsweise des Programms sehr oberflächlich erläutert. Dieser Teil ist nur für interessierte Personen gedacht und kann übersprungen werden.

### **Allgemeine Theorie**

Die Optimierung der Gangfolge fällt unter das [Botenproblem](#). Für eine perfekte Lösung ist damit keine Methode bekannt, welche das Problem in polynomialer Laufzeit löst. Grob gesagt kommt die Komplexität daher, dass der Zusteller an jeder Adresse die Entscheidung treffen muss, zu welcher noch nicht besuchten Adresse er als nächstes geht. Gibt es nur 5 Adressen, so hat er  $5! = 120$  Möglichkeiten, den Bezirk zu durchlaufen (Angenommen, er kann frei wählen, wo er startet), gibt es 10 Adressen, so hat er  $10! = 3,6$  Millionen Möglichkeiten, bei 15 sind es 1,3 Milliarden. Man bemerkt, wie schnell die Zahl ansteigt. Nun hat ein durchschnittlicher Bezirk aber eher 300-400 Adressen (Adressen, nicht individuelle Empfänger!). 350 Adressen wäre eine Zahl mit 740 Stellen. Von all diesen Möglichkeiten wird nicht nur die kürzeste und sicherste gesucht, sondern auch eine, bei der die Ablagen gut verteilt sind und keine Umwege darstellen. Eine normale „Brute Force“ Suche ist offensichtlich komplett unmöglich, jedenfalls innerhalb der Lebenszeit dieses Universums<sup>1</sup>.

1 Zum Vergleich: Die geschätzte Anzahl an Atomen im Universum ist eine Zahl mit weniger als 90 Stellen. Würden wir jedes einzelne Atom im Universum durch eine Kopie des Universums ersetzen, so müssten wir diesen Vorgang dreimal aufs Neue wiederholen (und zwar mit all den neuen Atomen und all den neuen Universen als Ersatz!), bevor wir so viele Atome wie Möglichkeiten hätten, einen herkömmlichen Bezirk mit 350 Adressen zu durchlaufen.

Glücklicherweise kommt natürlich nach einer Adresse nicht jede andere Adresse in Frage. Die meisten können von Beginn an ausgeschlossen werden. Trotzdem gibt die obige Rechnung eine Idee dafür, von wo gestartet wird, und dass auch ein paar simple Vereinfachungen bei weitem nicht ausreichend sind. Ebenfalls glücklicherweise fragt niemand nach der perfekten Route. Wenn der Computer eine vernünftige Lösung in akzeptabler Zeit ausspuckt, ist das gut genug. Unter anderen Randbedingungen als denen, die hier vorliegend, gibt es heutzutage auch bereits hervorragende [Algorithmen](#), welche extrem gute Lösungen für weitaus mehr „Adressen“ berechnen können (ohne Ablagen zu berücksichtigen).

### **1. Einlesen der Daten und Erstellung einer Straßenkarte**

Zunächst liest das Programm die Bezirksdatei ein, danach die zugehörige osm-Karte. Die osm-Karte wird in eine interne Straßenkarte verwandelt. Den Adressen werden Eingängen auf den Straßen zugeordnet (ein nichttrivialer Prozess, welcher für manche Bezirke länger dauert als jeder andere Schritt!). Die Karte wird auf den Bezirk zugeschnitten, und die Straßen werden in sinnvolle Segmente zerschnitten.

### **2. Erstellung der Eulerkarten**

Die Zuordnung jedes Straßensegments mit der Anzahl der Durchquerungen dieses Segments durch die fertige Route nenne ich Eulerkarte. Der Name entstand in Anlehnung an das [Eulerkreisproblem](#).

Das Programm entscheidet für jedes Straßensegment, ob die Anzahl, mit der es durchlaufen wird, gerade oder ungerade sein wird. Verschiedene Faktoren entscheiden danach, ob die tatsächliche Anzahl 0, 1, 2 oder 3-mal ist. Zum Abschluss wird berechnet, auf welcher Straßenseite der Durchlauf im optimalen Fall stattfindet. Schließlich werden die nach Straßenlänge kürzesten X gefundenen Eulerkarten weitergegeben, wobei „X“ die gewünschte Suchtiefe ist.

Eine große Anzahl an Tricks, wie das Erzwingen gerader Knotenpunkte, der Berechnung einer vermuteten Länge für noch nicht behandelte Segmente, die Vorsortierung der vermuteten Längen und der Abstandsmessung zwischen noch ungeraden Knoten, machen diesen Prozess überraschend schnell.

### **3. Erstellung der Routenkarten**

Eine Routenkarte ist eine Liste an miteinander verbundenen Pfaden und Knotenpunkten, wobei die Pfade bereits eine feste Richtung aufweisen. Aus jeder Eulerkarte entsteht eine Routenkarte.

Hierzu stellt sich das Programm zunächst vor, der Zusteller würde an jeder Kreuzung so abbiegen, dass es seinen Aufwand minimiert. Hierdurch entsteht keine zusammenhängende Route, sondern eine Reihe einzelner Eulerkreise. Für jeden dieser Kreise wird die optimale Richtung gefunden, sodass Einbahnstraßen und die richtige Straßenseite respektiert werden. Anschließend werden die Kreise an bestimmten Kreuzungen durch weitere „Kreuzungs-Kreise“ miteinander verknotet. Diese „Kreuzungs-Kreise“ berücksichtigen bereits strategisch die Lage der Ablagen. Die so miteinander verbundenen Kreise bilden die fertige Routenkarte. Die Berechnungszeit dieses Schritts ist vernachlässigbar.

#### **4. Erstellung der Route**

Zuletzt werden die Routenkarten nach der kürzesten Route, welche die Ablagen bestmöglich nutzt, durchsucht. Aufgrund der massiven Vorarbeit und ein paar weiterer Tricks ist dieser Prozess extrem schnell. So kann etwa der Status jeder Route in einer großen Zahl festgehalten werden, bei der jeder Bit angibt, ob der zugehörige Pfad bereits besucht wurde. Außerdem wird bei jeder Straßenüberquerung nicht nur der Effekt der Überquerung an sich berücksichtigt, sondern auch, wie sich alle noch nicht durchgeführten Überquerungen an der Kreuzung zwangsweise ändern müssen. Die fertige Route wird zum Schluss ausgelesen und analysiert.

### **Limitationen**

Das Programm hat die folgenden bekannten größeren Limitationen:

#### **Dies ist ein Hobbyprojekt**

Auch wenn das Programm tatsächlich sehr gut funktioniert, werden im Ablauf mehrere Annahmen gemacht, die für manche spezielle Bezirke schlichtweg nicht gelten. Das Fehlen einer grafischen Oberfläche limitiert das Programm ebenfalls.

#### **Limitation aufs Stadtgebiet**

Das Programm wurde ausschließlich im Stadtgebiet durchdacht und getestet. In ländlichen Regionen gibt es keinerlei Garantie auf die korrekte Funktionsweise.

#### **Hauseingänge werden erraten**

Die Lage von Hauseingängen ist für die meisten Adressen leicht zu erraten, für andere höchst nichttrivial. Das Programm benutzt eine Reihe von geometrischen Funktionen um die korrekte Lage zu schätzen. Trotzdem gibt es Fehler, vor allem bei Gebäuden mit mehreren Eingängen. Ferner erwarten die Funktionen „herkömmliche Adressformate“. Die „[Mannheimer Quadrate](#)“ sind nur ein Beispiel für ein inkompatibles Schema.

#### **Postmenge wird geschätzt**

Aufgrund fehlender Daten wird die Menge an Post pro Adresse über die Art des Gebäudes, dessen Etagenanzahl und Grundrissgröße geschätzt. Hierbei entstehen ohne Frage Fehler.

#### **Konsistenz**

Kleine Änderungen der zuzustellenden Adressen können zu großen Änderungen der Gangfolge führen. Dies bedeutet, dass der Zusteller möglicherweise seinen Bezirk komplett anders laufen muss, nur weil sich zwei Straßen geändert haben.

#### **Kleines Betriebsmittel**

Das Programm geht von einem kleinem Betriebsmittel aus, welches im Notfall auch auf dem Bürgersteig oder durch nicht allzu kleine Gassen fahren kann. Große Betriebsmittel haben auch andere Vorzüge beim Abbiegen an Kreuzungen als kleine. Die berechneten Gangfolgen sind für Automobile ungeeignet!

### **Open Street Map**

Das Programm ist maximal nur so gut wie die benutzte Karte. „Open Street Map“ ist gut, hat aber auch einige Fehler. Für diese Fälle gilt dann: Wo Müll reinkommt, kann auch nur Müll rauskommen.

### **Lizenzen und ähnliches**

Dieses Projekt benutzt Daten des „[Open-Street-Map](#)“-Projekts. Die Daten sind erhältlich unter der [Open database Lizenz](#).

Das Projekt benutzt zur Darstellung der Daten in der Konsole „[Spectre.Console](#)“ von Patrik Svensson, Phil Scott, Nils Andresen und Cédric Luthi, erhältlich unter der [MIT-Lizenz](#).

Das Projekt benutzt zuletzt die „Priority Queue“ von Daniel Pflughoeft, erhältlich unter der [MIT-Lizenz](#).

Während der absolute Großteil des Codes zu Übungszwecken ohne die Hilfe von künstlicher Intelligenz geschrieben wurde, hat [Claude](#) zum Abschluss beim Finden und Beheben von ein paar Bugs sehr geholfen.

**Danke an alle!**